

Documentación: Gestión de Rutas en React con React Router

La gestión de rutas es fundamental en las aplicaciones de una sola página (*Single Page Applications - SPA*) de React. Permite que la URL cambie sin recargar toda la página y muestre un contenido diferente según la dirección.

Utilizaremos la librería más popular y robusta para este fin: **React Router DOM**.

1. 🛠 Instalación y Configuración

Antes de usar las rutas, es necesario instalar la librería en tu proyecto de React (usando npm o yarn).

Bash

```
# Comando de instalación
npm install react-router-dom
# o
yarn add react-router-dom
```

2. Componentes Clave de React Router

React Router se basa en componentes para definir la navegación y la estructura de la aplicación. Los más importantes son:

Componente	Uso Principal
BrowserRouter	Envuelve toda la aplicación. Usa la API de historial del navegador (el <i>History API</i>) para mantener la interfaz de usuario sincronizada con la URL. Debe ser el componente raíz de tu aplicación.
Routes	Contenedor principal donde se definen todas las rutas individuales. Solo renderiza la primera <Route> que coincida con la URL actual.
Route	Mapea un componente específico a una <code>path</code> (ruta o URL).
Link	Reemplaza la etiqueta <a> HTML tradicional. Previene la recarga completa de la página y usa el enrutamiento interno de React.
useNavigate	Un <i>Hook</i> utilizado para la navegación programática (por ejemplo,

Componente	Uso Principal
	después de un envío de formulario exitoso).

3. Configuración Básica de las Rutas

La configuración de rutas se hace típicamente en el archivo principal (`App.js` o `main.jsx`).

Ejemplo de Estructura en `App.js`

JavaScript

```
// 1. Importar los componentes necesarios
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

// 2. Componentes de las páginas
function PaginaInicio() {
  return <h2>□ Inicio</h2>;
}

function PaginaAcerca() {
  return <h2>■ □ Acerca de Nosotros</h2>;
}

function PaginaContacto() {
  return <h2>□ Contáctanos</h2>;
}

function PaginaNoEncontrada() {
  return <h2>404 - Página no encontrada</h2>;
}

// 3. Componente principal que define la estructura
function App() {
  return (
    // 4. El BrowserRouter debe envolver todo
    <BrowserRouter>
      <h1>Mi Aplicación con Rutas</h1>

      {/* 5. Componente de Navegación */}
      <nav>
        <Link to="/">Inicio</Link> |{' '}
        <Link to="/acerca">Acerca de</Link> |{' '}
        <Link to="/contacto">Contacto</Link>
      </nav>

      <hr />

      {/* 6. Contenedor de las Rutas */}
      <Routes>
        {/* Route 1: Ruta raíz */}
        <Route path="/" element={<PaginaInicio />} />
      </Routes>
    </BrowserRouter>
  );
}
```

```

    {/* Route 2: Ruta estática */}
    <Route path="/acerca" element={<PaginaAcerca />} />

    {/* Route 3: Otra ruta estática */}
    <Route path="/contacto" element={<PaginaContacto />} />

    {/* Route 4: Ruta 404 (Catch-all) */}
    {/* El path="*" coincide con cualquier URL que no haya
coincido antes */}
    <Route path="*" element={<PaginaNoEncontrada />} />
  </Routes>
</BrowserRouter>
);
}

// export default App;

```

Explicación:

- **BrowserRouter:** Permite que la aplicación sea consciente de las rutas.
- **Link:** Genera enlaces navegables que cambian la URL sin recargar la página.
- **Routes:** Dentro de este contenedor, React Router busca una coincidencia de URL.
- **Route path="/" element={<Componente />}:** Cuando la URL coincide con /, renderiza el componente.
- **path="*":** Es la última ruta; si ninguna de las anteriores coincide, renderiza el componente 404.

4. Rutas Dinámicas y Parámetros

A menudo, necesitas una ruta que cambie según un identificador (ID) para mostrar el detalle de un recurso (por ejemplo, el detalle de un producto o de un usuario).

Se define un **parámetro de URL** usando dos puntos (:).

Ejemplo: Ruta de Detalle de Producto

Definición de la Ruta (en App.js):

JavaScript

```

// ... dentro del componente Routes
<Route path="/producto/:idProducto" element={<PaginaProductoDetalle
/>} />

```

Componente para Leer el Parámetro:

Para acceder al valor del parámetro (`idProducto` en este caso), utilizamos el Hook `useParams`.

JavaScript

```

import React from 'react';
import { useParams } from 'react-router-dom';

```

```
function PaginaProductoDetalle() {
  // Extrae los parámetros definidos en el path (ej: :idProducto)
  const parametros = useParams();
  const id = parametros.idProducto;

  // Alternativamente, se puede desestructurar directamente:
  // const { idProducto } = useParams();

  // Aquí realizarías una llamada a la API usando el 'id'
  // useEffect(() => { ... }, [id]);

  return (
    <div>
      <h3>□ Detalle del Producto</h3>
      <p>ID del Producto solicitado: <strong>{id}</strong></p>
      <p>Aquí se cargaría la información específica del producto {id}</p>
    </div>
  );
}
```

Funcionamiento: Si el usuario navega a /producto/123, el valor de idProducto será "123".

5. Navegación Programática (useNavigate)

Para redirigir al usuario después de una acción (como iniciar sesión o enviar un formulario), usamos el Hook `useNavigate`.

Ejemplo: Redirección después de Iniciar Sesión

JavaScript

```
import React, { useState } => 'react';
import { useNavigate } from 'react-router-dom';

function ComponenteLogin() {
  // Inicializamos el Hook
  const navigate = useNavigate();
  const [usuario, setUsuario] = useState('');
  const [password, setPassword] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();

    // 1. Lógica de autenticación (simulación)
    if (usuario === 'admin' && password === '1234') {
      console.log('Inicio de sesión exitoso.');
```

// 2. Navegación programática: Redirigir a la página de inicio
 // El argumento es la ruta a la que se desea ir
 navigate('/');

// O para reemplazar la entrada en el historial (el usuario no
 puede volver con el botón atrás)
 // navigate('/', { replace: true });

```

    } else {
      alert('Credenciales inválidas.');
```

```

    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <h3>🔑 Iniciar Sesión</h3>
      <input
        type="text"
        placeholder="Usuario"
        value={usuario}
        onChange={(e) => setUsuario(e.target.value)}
      />
      <input
        type="password"
        placeholder="Contraseña"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <button type="submit">Entrar</button>
    </form>
  );
}

```

Explicación: Después de la lógica de negocio (autenticación exitosa), la llamada `Maps ( '/' )` cambia la URL y renderiza el componente asignado a la ruta raíz.

Ejemplo Avanzado: Rutas Anidadas (Nested Routes) en React Router

Las rutas anidadas permiten que los componentes hijos (*children*) definan sus propios caminos relativos, basándose en la ruta del componente padre.

1. El Concepto

Imagina que tienes una sección `/dashboard`. Dentro de este *dashboard*, quieres tres vistas distintas: `/dashboard/perfil`, `/dashboard/informes` y `/dashboard/configuracion`.

En lugar de definir todas estas rutas planas en el archivo principal (`App.js`), defines la ruta `/dashboard` y dejas que el componente asociado a ella (`DashboardLayout`) se encargue de manejar sus propias subrutas.

2. 🛠 Implementación con Outlet

La clave para las Rutas Anidadas es el componente `Outlet`.

- El componente **Padre** (ej: `DashboardLayout`) renderiza el componente `Outlet`.
- El `Outlet` es donde React Router inyectará y renderizará el contenido del componente **Hijo** que coincida con la subruta.

Paso 1: Definir los Componentes de las Páginas Anidadas

JavaScript

```
// Componentes Hijos
function PaginaPerfil() {
  return <h4>Mi Perfil de Usuario</h4>;
}

function PaginaInformes() {
  return <h4>Informes Mensuales</h4>;
}

function PaginaConfiguracion() {
  return <h4>Ajustes de la Aplicación</h4>;
}
```

Paso 2: Crear el Componente Layout (Padre)

Este componente contiene la estructura estática (el menú lateral) y el componente Outlet donde se mostrarán los hijos.

JavaScript

```
import { Outlet, Link } from 'react-router-dom';

function DashboardLayout() {
  return (
    <div style={{ display: 'flex', border: '1px solid #ccc', padding: '10px' }}>

      {/* Estructura Estática (Barra Lateral/Menú) */}
      <nav style={{ paddingRight: '20px', borderRight: '1px solid #eee' }}>
        <h3>Menú Dashboard</h3>
        <ul>
          <li>
            {/* El Link es relativo al path del padre (/dashboard) */}
            <Link to="perfil">Ver Perfil</Link>
          </li>
          <li>
            <Link to="informes">Ver Informes</Link>
          </li>
          <li>
            <Link to="configuracion">Configuración</Link>
          </li>
        </ul>
      </nav>

      {/* □ El OUTLET es donde se renderizará el contenido de las subrutas */}
      <main style={{ paddingLeft: '20px', flexGrow: 1 }}>
        <Outlet />
      </main>

    </div>
  );
}
```

Paso 3: Configurar las Rutas en App.js

Utilizamos la etiqueta `<Route>` de cierre automático y anidamos las subrutas dentro del padre.

JavaScript

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
// Importar todos los componentes definidos arriba...

function AppConAnidacion() {
  return (
    <BrowserRouter>
      <Routes>
        {/* Rutas no anidadas (ej. Inicio) */}
        <Route path="/" element={<h1>Página Principal</h1>} />

        {/* 1. RUTA PADRE: Su componente DashboardLayout contiene el Outlet */}
        <Route path="/dashboard" element={<DashboardLayout />}>

          {/* 2. RUTAS HIJAS: Se renderizan DENTRO del Outlet del padre. */}

          {/* Ruta por defecto del Dashboard (ej: /dashboard) */}
          <Route index element={<h4>Bienvenido al Dashboard.
            Selecciona una opción.</h4>} />

          {/* Ruta: /dashboard/perfil */}
          <Route path="perfil" element={<PaginaPerfil />} />

          {/* Ruta: /dashboard/informes */}
          <Route path="informes" element={<PaginaInformes />} />

          {/* Ruta: /dashboard/configuracion */}
          <Route path="configuracion" element={<PaginaConfiguracion
            />} />

          {/* Sub-ruta 404 (Si estás en /dashboard/algo-inexistente) */}
          <Route path="*" element={<h4>Sub-Página no encontrada en
            Dashboard</h4>} />

        </Route>

        {/* Ruta 404 general (Si la URL es completamente desconocida) */}
        <Route path="*" element={<h2>404 - Página Principal No
            Encontrada</h2>} />

      </Routes>
    </BrowserRouter>
  );
}
```

Ventajas de las Rutas Anidadas

1. **Mantenimiento de Layouts:** El `DashboardLayout` solo se monta y desmonta una vez. Cuando cambias de `/dashboard/perfil` a `/dashboard/informes`, solo se destruye y recrea la vista del componente hijo, manteniendo el estado de la barra lateral estático.

2. **Modularidad:** El componente padre solo necesita saber que existen subrutas; las subrutas mismas pueden ser definidas en archivos separados, manteniendo el código limpio.
3. **Rutas Relativas:** Usar *paths* sin la barra inicial (ej: `path="perfil"` en lugar de `path="/dashboard/perfil"`) asegura que las rutas hijas son automáticamente prefijadas por el *path* del padre.